



Universidad Nacional de Moquegua

Facultad de Ingeniería

Escuela Profesional de Ingeniería de Sistemas e Informática.

INGENIERÍA DE SOFTWARE

DOCUMENTO DE DISEÑO DE SOFTWARE - SDD 2

Docente:

DR. MORALES GONZALES, RUSSO ALEXANDER

Alumno:

Rojas Quijano, Gersael Mathias

ILO

2026



DOCUMENTO DE DISEÑO DE SOFTWARE – SDD 1

1.1 INFORMACIÓN GENERAL

Sistema Web de Gestión Visual de Inventario, Ventas y Control de Anaqueles para Minimarkets.

1.2 Versión

10.0.0

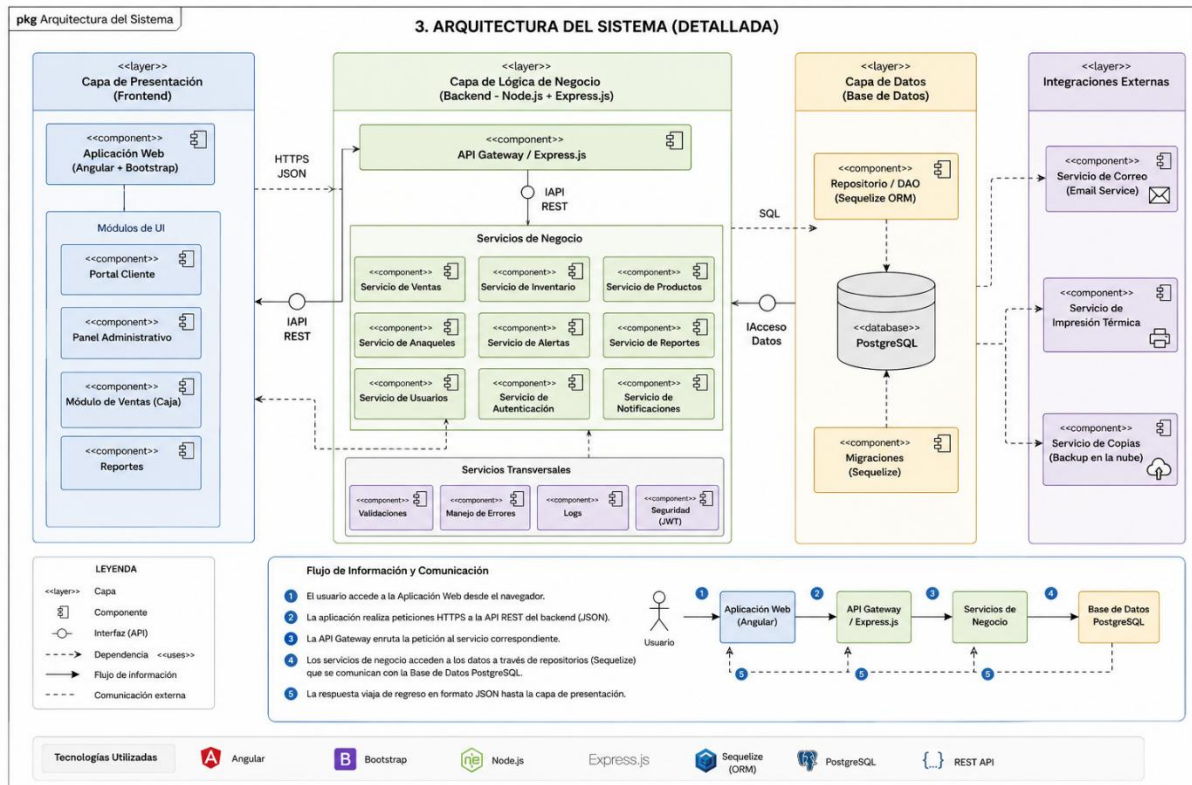
1.3 Autor

Gersael Mathias Rojas Quijano

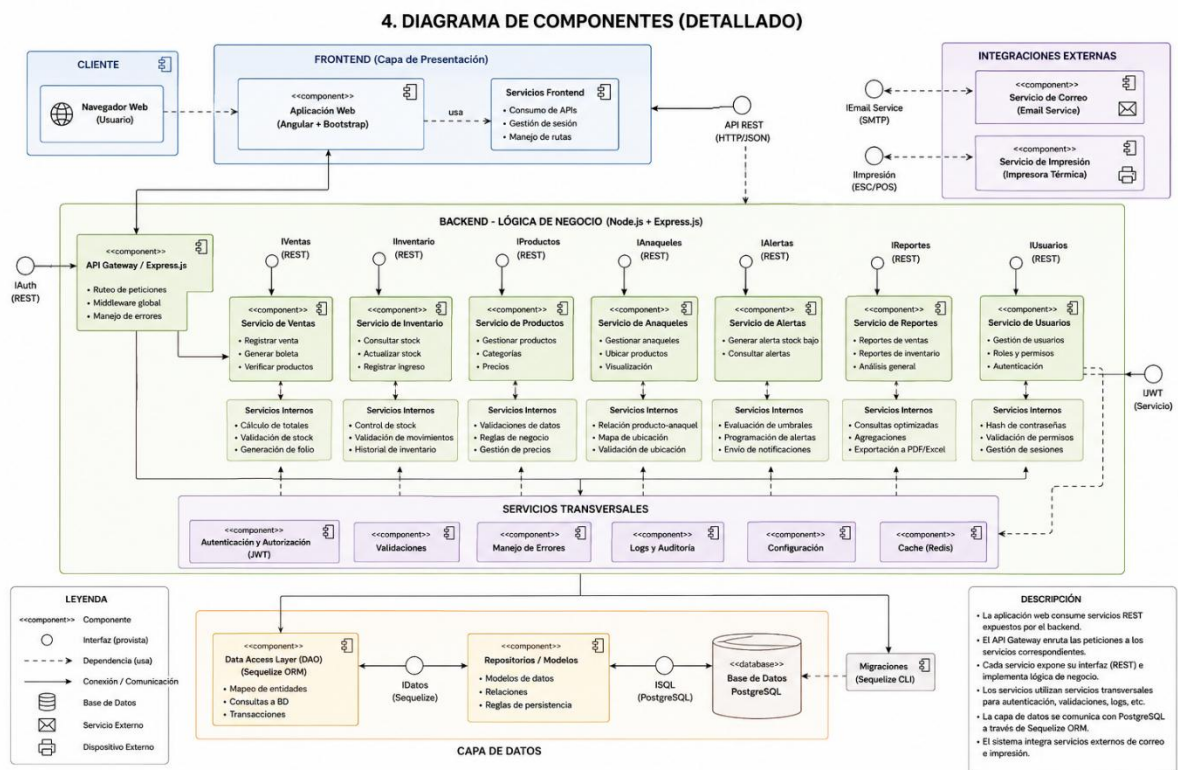
2. CAMBIOS RESPECTO A SDD 1

Elemento	Mejora aplicada
Arquitectura	Se refinó la arquitectura preliminar del SDD 1 incorporando una estructura por capas: presentación, lógica de negocio y datos. Además, se precisó la tecnología utilizada en cada capa: Angular y Bootstrap para el frontend, Node.js con Express.js para el backend y PostgreSQL para la base de datos. También se detalló el flujo de comunicación mediante API REST.
Componentes	Se mejoró el diagrama de componentes agregando mayor precisión en los módulos funcionales del sistema, sus interfaces, dependencias y servicios internos. Se diferencian los módulos principales como Portal Cliente, Ventas, Inventario, Anaqueles, Alertas y Reportes, además de componentes transversales como Gestión de Usuarios y Seguridad.
Datos	Se refinó el modelo de datos inicial, incorporando un DER más completo y un modelo relacional con tablas, campos, tipos de datos, claves primarias y claves foráneas. También se considera la normalización para asegurar que la base de datos sea coherente, organizada y lista para implementación en PostgreSQL.
Red / despliegue	Se mejoró el despliegue preliminar incorporando un diseño lógico de red, donde se representa la comunicación entre cliente, servidor de aplicaciones, servidor de base de datos y servicios externos. Se mantiene una arquitectura cliente-servidor clara, evitando detalles físicos innecesarios y enfocándose en la infraestructura lógica del sistema.

3. ARQUITECTURA DEL SISTEMA

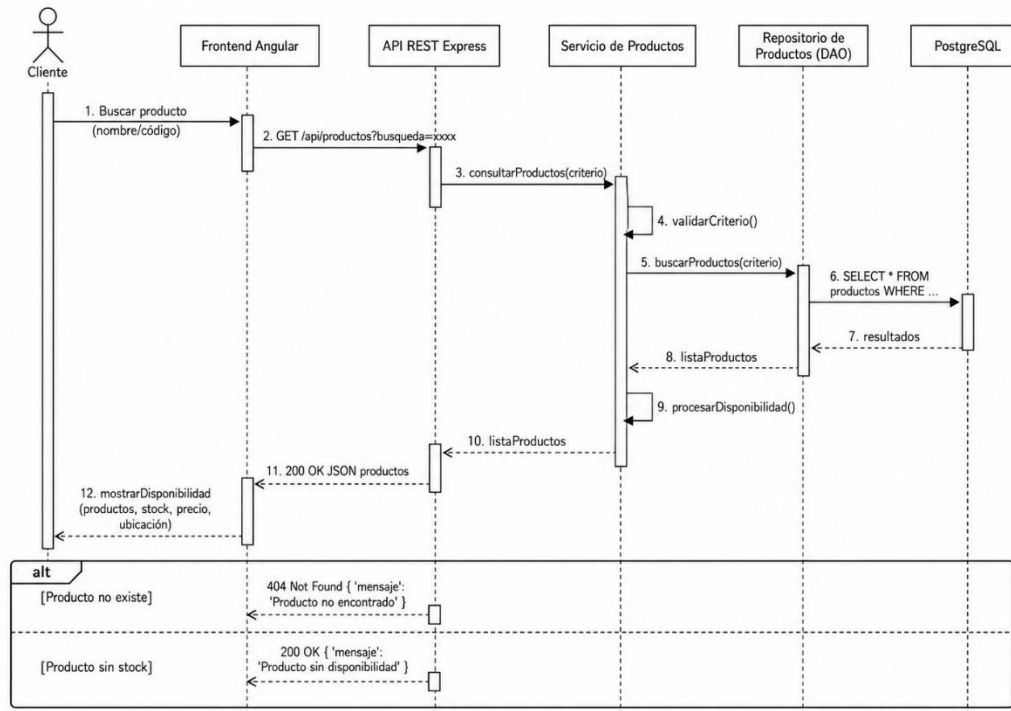


4. DIAGRAMA DE COMPONENTES

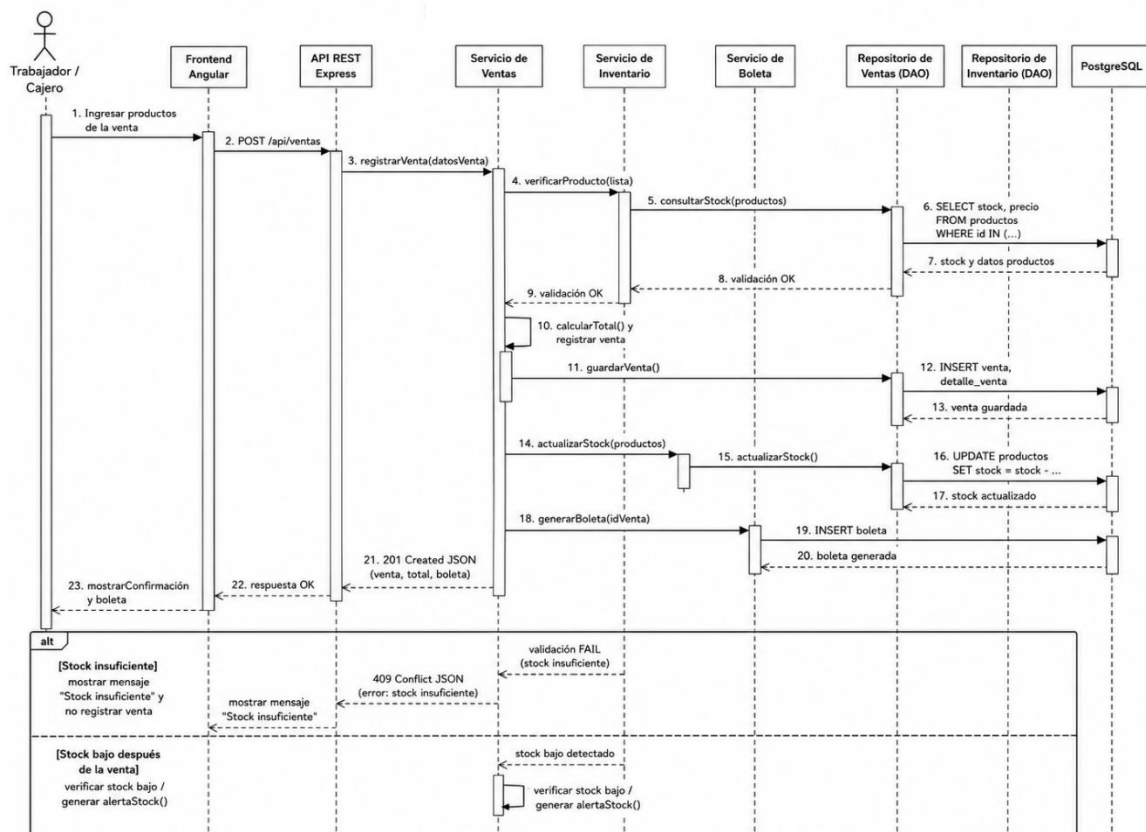


5. DIAGRAMAS DE SECUENCIA

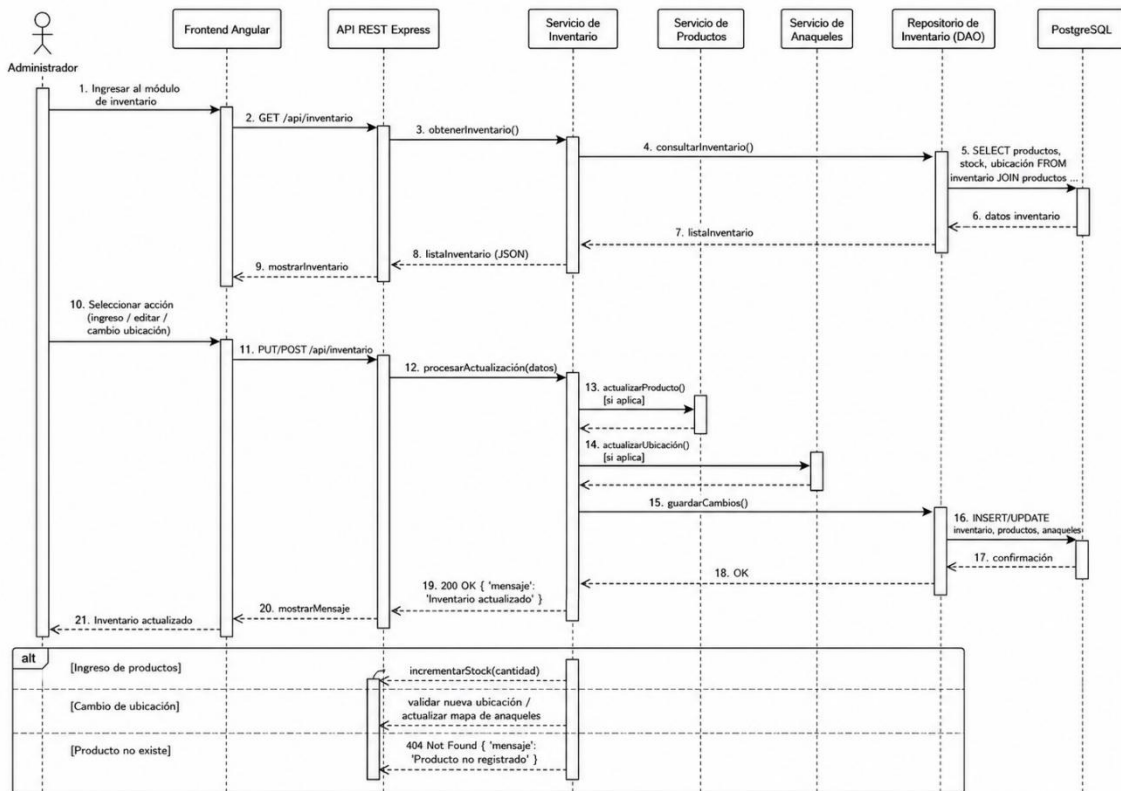
5.1. CU02 – Consultar disponibilidad



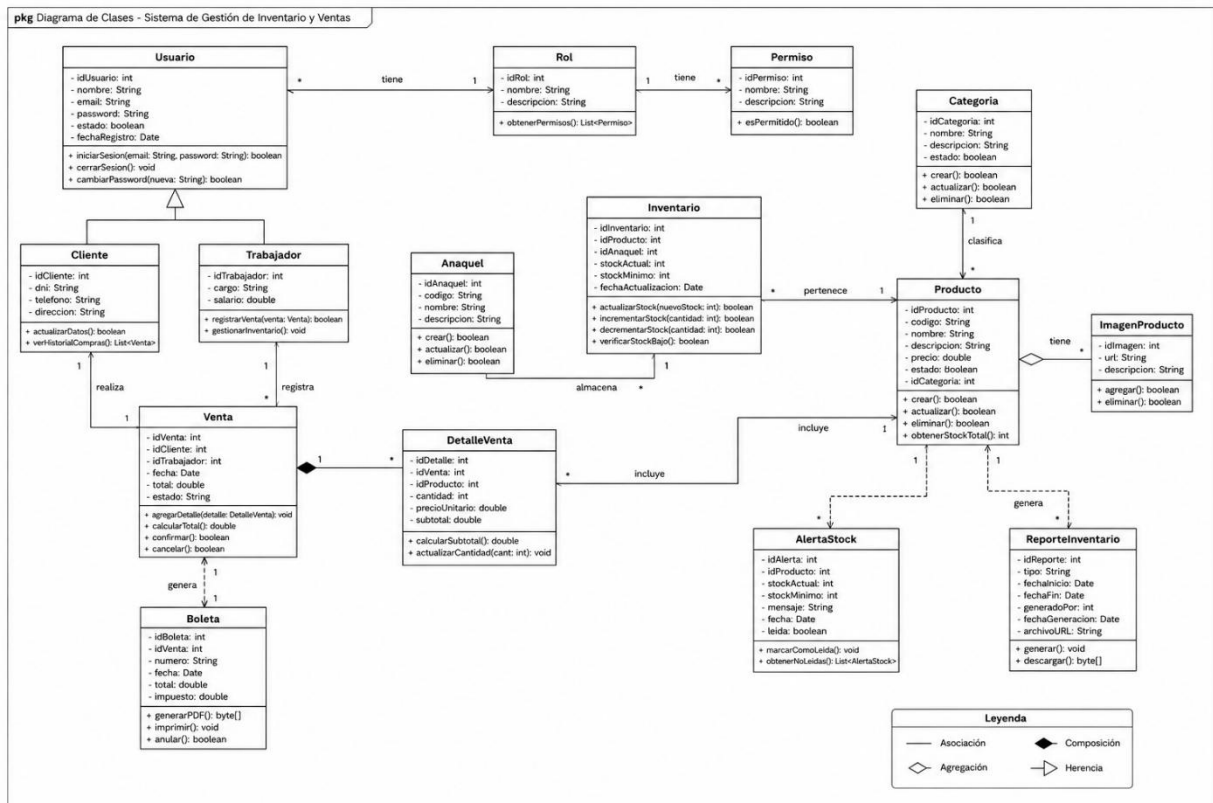
5.2. CU05 – Register venta



5.3. CU05 – Gestionar inventario

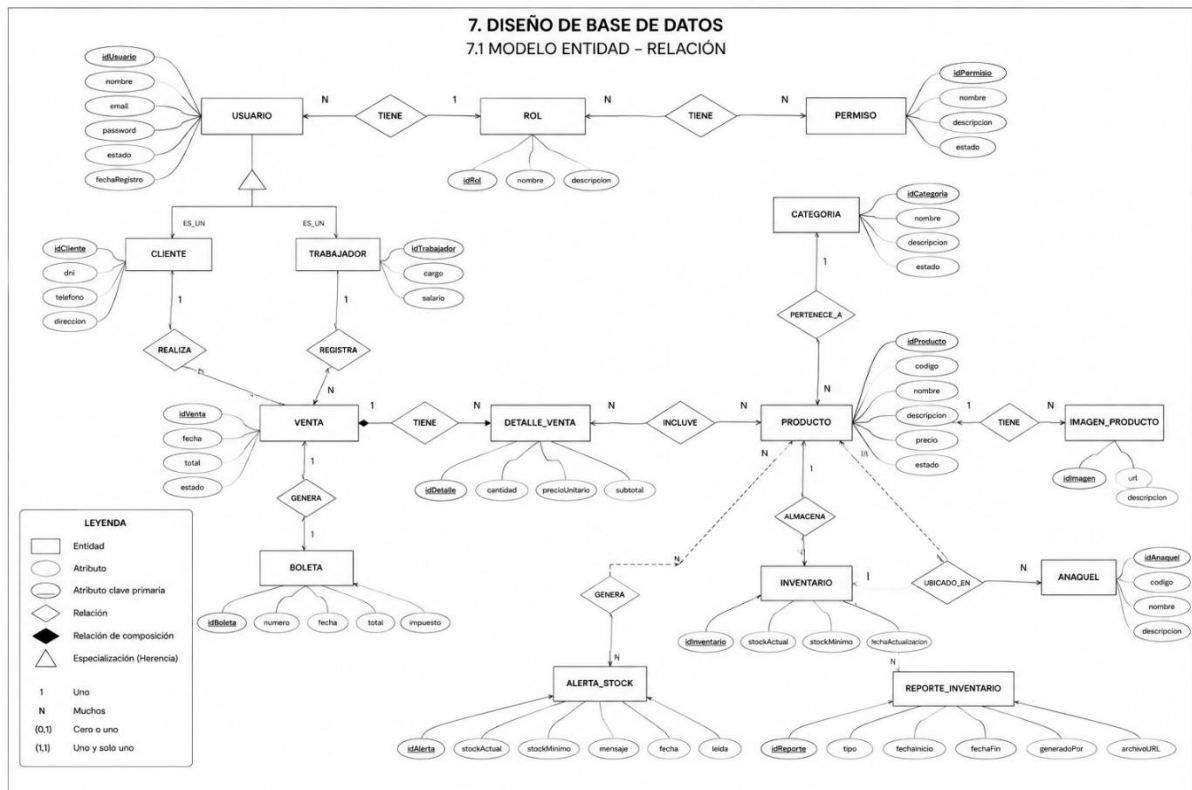


6. DIAGRAMA DE CLASES (COMPLETO)



7. DISEÑO DE BASE DE DATOS

7.1 MODELO ENTIDAD-RELACIÓN



7.2 MODELO RELACIONAL

Tabla	Campo	Tipo de dato	PK/FK
rol	id_rol	SERIAL	PK
	nombre	VARCHAR(50)	
	descripcion	VARCHAR(150)	
permiso	id_permiso	SERIAL	PK
	nombre	VARCHAR(80)	
	descripcion	VARCHAR(150)	
	estado	BOOLEAN	
rol_permiso	id_rol	INT	PK / FK
	id_permiso	INT	PK / FK
usuario	id_usuario	SERIAL	PK
	id_rol	INT	FK
	nombre	VARCHAR(80)	
	email	VARCHAR(120)	
	password	VARCHAR(255)	
	estado	BOOLEAN	
	fecha_registro	TIMESTAMP	



Tabla	Campo	Tipo de dato	PK/FK
cliente	id_cliente	SERIAL	PK
	id_usuario	INT	FK
	dni	VARCHAR(15)	
	telefono	VARCHAR(20)	
	direccion	VARCHAR(150)	
trabajador	id_trabajador	SERIAL	PK
	id_usuario	INT	FK
	cargo	VARCHAR(80)	
	salario	NUMERIC(10,2)	
categoria	id_categoria	SERIAL	PK
	nombre	VARCHAR(80)	
	descripcion	VARCHAR(150)	
	estado	BOOLEAN	
anaquel	id_anaquel	SERIAL	PK
	codigo	VARCHAR(30)	
	nombre	VARCHAR(80)	
	descripcion	VARCHAR(150)	
producto	id_producto	SERIAL	PK
	id_categoria	INT	FK
	id_anaquel	INT	FK
	codigo	VARCHAR(50)	
	nombre	VARCHAR(100)	
	descripcion	VARCHAR(200)	
	precio	NUMERIC(10,2)	
imagen_producto	id_imagen	SERIAL	PK
	id_producto	INT	FK
	url	VARCHAR(255)	
	descripcion	VARCHAR(150)	
inventario	id_inventario	SERIAL	PK
	id_producto	INT	FK
	id_anaquel	INT	FK
	stock_actual	INT	
	stock_minimo	INT	
	fecha_actualizacion	TIMESTAMP	
venta	id_venta	SERIAL	PK
	id_cliente	INT	FK
	id_trabajador	INT	FK

Tabla	Campo	Tipo de dato	PK/FK
	fecha	TIMESTAMP	
	total	NUMERIC(10,2)	
	estado	VARCHAR(30)	
detalle_venta	id_detalle	SERIAL	PK
	id_venta	INT	FK
	id_producto	INT	FK
	cantidad	INT	
	precio_unitario	NUMERIC(10,2)	
	subtotal	NUMERIC(10,2)	
boleta	id_boleta	SERIAL	PK
	id_venta	INT	FK
	numero	VARCHAR(30)	
	fecha	TIMESTAMP	
	total	NUMERIC(10,2)	
	impuesto	NUMERIC(10,2)	
alerta_stock	id_alerta	SERIAL	PK
	id_producto	INT	FK
	stock_actual	INT	
	stock_minimo	INT	
	mensaje	VARCHAR(200)	
	fecha	TIMESTAMP	
	leida	BOOLEAN	
reporte_inventario	id_reporte	SERIAL	PK
	generado_por	INT	FK
	tipo	VARCHAR(50)	
	fecha_inicio	DATE	
	fecha_fin	DATE	
	fecha_generacion	TIMESTAMP	
	archivo_url	VARCHAR(255)	

7.3 NORMALIZACIÓN

7.3.1. Primera Forma Normal (1FN)

El modelo cumple con la Primera Forma Normal porque todos los campos almacenan valores atómicos, es decir, no existen listas ni grupos repetidos dentro de una misma columna. Por ejemplo, los productos no almacenan varias categorías en un solo campo, sino que cada producto se relaciona con una categoría mediante id_categoria.

7.3.2. Segunda Forma Normal (2FN)

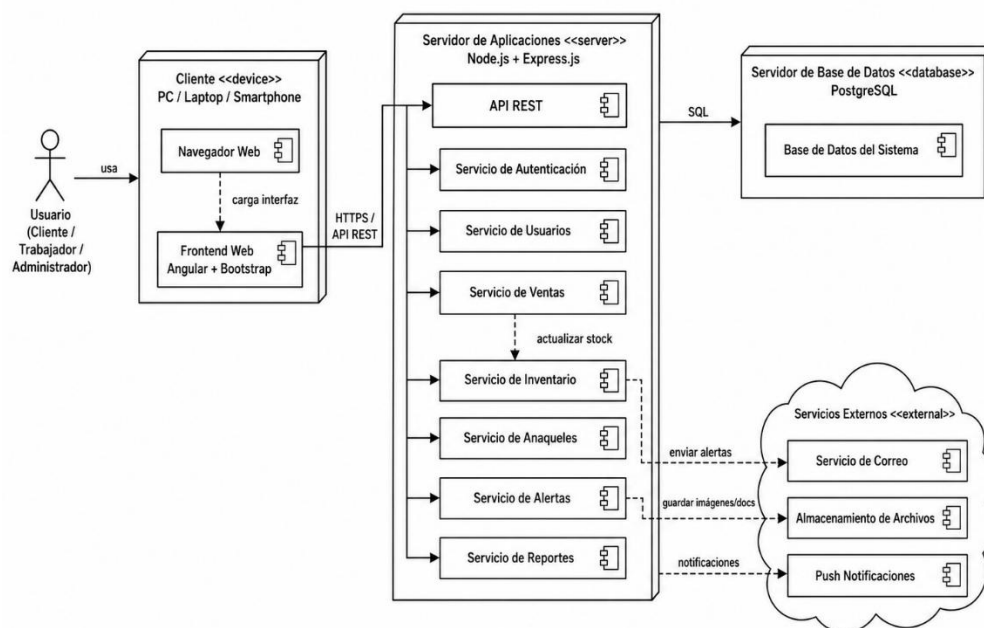
El modelo cumple con la Segunda Forma Normal porque cada atributo depende completamente de la clave primaria de su tabla. Por ejemplo, en la tabla detalle_venta, los campos cantidad, precio_unitario y subtotal dependen del registro del detalle de venta y no de otra entidad externa. Además, las relaciones muchos a muchos se resuelven mediante tablas intermedias, como rol_permiso.

7.3.3 Tercera Forma Normal (3FN)

El modelo cumple con la Tercera Forma Normal porque no existen dependencias transitivas entre campos no clave. Los datos de usuarios, roles, permisos, productos, categorías, anaqueles, ventas, boletas, inventario y alertas se encuentran separados en tablas independientes. Por ejemplo, la información de la categoría no se repite dentro de la tabla producto, sino que se relaciona mediante id_categoria.

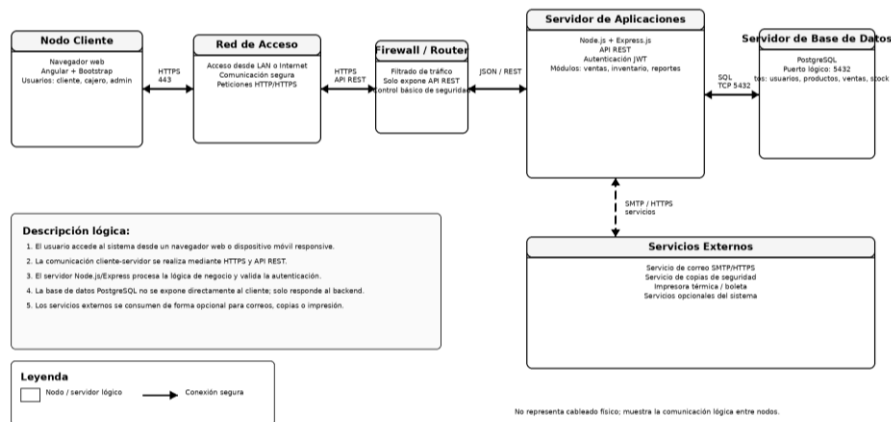
8. DIAGRAMA DE DESPLIEGUE

DIAGRAMA DE DESPLIEGUE - Sistema Web de Gestión Visual de Inventario, Ventas y Control de Anaqueles



9. DISEÑO DE RED

9.1 DIAGRAMA DE RED



9.2 TOPOLOGÍA DE RED

9.2.1. Topología seleccionada Cliente-servidor.

Se selecciona esta topología porque los usuarios acceden desde un navegador y consumen servicios centralizados del backend mediante API REST.

9.2.2. Justificación técnica.

Centraliza la lógica del sistema en el servidor backend Node.js + Express.js.

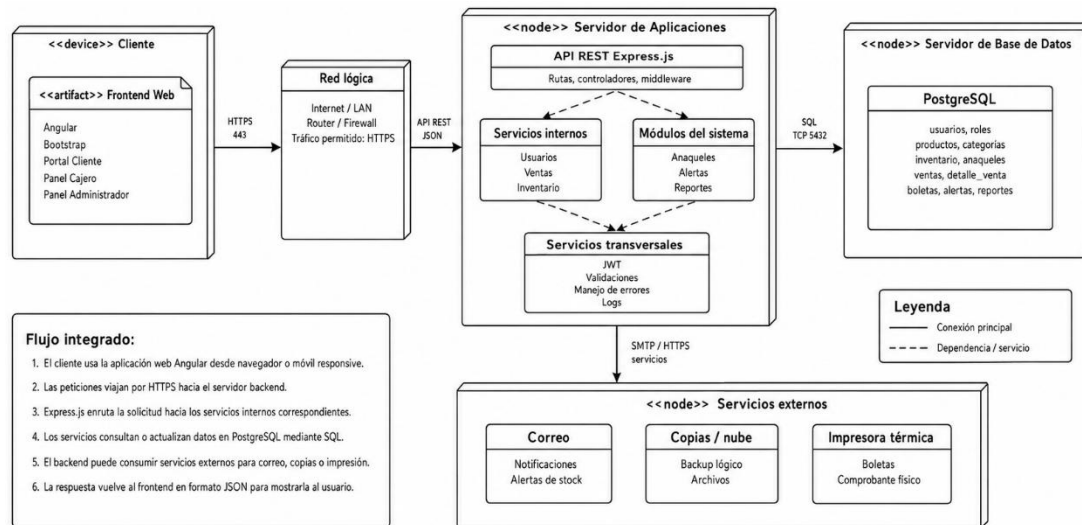
Permite acceso desde navegador web o móvil responsive mediante Angular + Bootstrap.

Protege la base de datos PostgreSQL, ya que no se expone directamente al cliente.

Facilita autenticación, roles, permisos, mantenimiento e integración con servicios externos.

Es suficiente para el alcance del minimarket y evita complejidad innecesaria.

9.3 DIAGRAMA DE DESPLIEGUE MÁS RED



10. RELACIÓN COMPLETA DEL SISTEMA

Requisito	Caso de uso	Componente	Clase	Tabla
RF01: El sistema debe permitir buscar productos por nombre o categoría.	CU01 – Buscar productos	Portal Cliente	Producto / Categoría	productos / categorías
RF02: El sistema debe mostrar la disponibilidad de productos en tiempo real.	CU02 – Consultar disponibilidad	Portal Cliente / Gestión de Inventario	Producto / Inventario	productos / inventario
RF03: El sistema debe mostrar el precio actualizado de los productos.	CU03 – Visualizar precio	Portal Cliente / Gestión de Inventario	Producto	productos
RF04: El sistema debe mostrar la ubicación del producto dentro de los anaques.	CU04 – Visualizar ubicación	Portal Cliente / Gestor Visual de Anaques	Producto / Anaquel	productos / anaques
RF05: El trabajador o cajero debe	CU05 – Registrar venta	Módulo de Ventas	Venta / DetalleVenta / Usuario	ventas / detalle_ventas / usuarios

Requisito	Caso de uso	Componente	Clase	Tabla
poder registrar una venta.				
RF06: El sistema debe permitir verificar productos antes de confirmar una venta.	CU06 – Verificar productos	Módulo de Ventas / Gestión de Inventario	Producto / DetalleVenta	productos / detalle_ventas / inventario
RF07: El sistema debe generar automáticamente una boleta de venta.	CU07 – Generar boleta	Módulo de Ventas	Boleta / Venta	boletas / ventas
RF08: El sistema debe actualizar automáticamente el stock después de cada venta.	CU08 – Actualizar stock	Gestión de Inventario	Producto / MovimientoInventario	productos / inventario / movimientos_inventario
RF09: El sistema debe permitir consultar alertas de stock bajo.	CU09 – Consultar alertas de stock	Módulo de Alertas	AlertaStock / Producto	alertas_stock / productos
RF10: El administrador debe poder gestionar el inventario general.	CU10 – Gestionar inventario	Gestión de Inventario	Producto / Categoría / MovimientoInventario	productos / categorias / inventario / movimientos_inventario
RF11: El administrador debe poder gestionar productos del sistema.	CU11 – Gestionar productos	Gestión de Inventario	Producto / Categoría	productos / categorias
RF12: El administrador debe registrar ingresos de	CU12 – Registrar ingreso de productos	Gestión de Inventario	Producto / MovimientoInventario	productos / inventario / movimientos_inventario

Requisito	Caso de uso	Componente	Clase	Tabla
productos al inventario.				
RF13: El sistema debe permitir gestionar anaqueles y ubicación de productos.	CU13 – Gestionar anaqueles	Gestor Visual de Anaqueles	Anaquel / Producto	anaqueles / productos
RF14: El sistema debe permitir visualizar reportes de ventas e inventario.	CU14 – Visualizar reportes	Reportes y Análisis	Venta / DetalleVenta / Producto / MovimientoInventario	ventas / detalle_ventas / productos / movimientos_inventario
RF15: El sistema debe generar alertas cuando un producto alcance el stock mínimo.	CU15 – Generar alerta de stock bajo	Módulo de Alertas / Gestión de Inventario	AlertaStock / Producto	alertas_stock / productos

11. DECISIONES DE DISEÑO

Decisión	Justificación
Uso de arquitectura en capas	Se separan responsabilidades entre presentación, lógica de negocio y datos. Esto facilita el mantenimiento, la escalabilidad, la reutilización de componentes y la corrección de errores sin afectar todo el sistema.
Uso de arquitectura cliente-servidor	El sistema será usado desde navegadores web por clientes, trabajadores y administradores, mientras que la lógica principal se ejecutará en el servidor. Esto centraliza la información y evita instalaciones independientes en cada equipo.
Uso de API REST	Permite que el frontend se comuniquen con el backend mediante peticiones HTTP y respuestas JSON. Además, facilita que en el futuro el sistema pueda ser consumido por una aplicación móvil u otros clientes.
Uso de frontend responsive	La interfaz debe funcionar correctamente en computadoras, laptops, tablets y celulares. Esto es necesario porque el

Decisión	Justificación
	cliente puede consultar productos desde un celular y el trabajador puede operar el sistema desde una PC de caja.
Uso de Angular + Bootstrap	Angular permite construir una interfaz modular basada en componentes, vistas y servicios. Bootstrap facilita el diseño responsive y acelera la creación de formularios, tablas, botones y paneles visuales.
Uso de backend con Node.js + Express.js	Node.js con Express.js permite desarrollar una API REST ligera, rápida y organizada mediante rutas, controladores y servicios. Es adecuado para procesar ventas, inventario, alertas y reportes.
Uso de base de datos relacional PostgreSQL	El sistema maneja datos estructurados y relacionados, como usuarios, roles, productos, ventas, boletas, anaqueles y alertas. PostgreSQL permite mantener integridad, relaciones y consultas consistentes.
Separación entre servidor de aplicación y base de datos	El cliente no accede directamente a la base de datos. Solo el backend puede consultar o modificar información, lo que mejora la seguridad y permite validar reglas de negocio antes de guardar cambios.
Uso de autenticación y roles	El sistema cuenta con perfiles diferentes: cliente, trabajador/cajero y administrador. Por ello, se requiere controlar permisos para evitar que un usuario acceda a funciones que no le corresponden.
Actualización automática de stock	Cada venta registrada descuenta automáticamente las cantidades vendidas. Esto reduce errores manuales y mantiene el inventario actualizado para consultas, reportes y alertas.
Generación automática de alertas de stock bajo	Cuando un producto alcanza su stock mínimo, el sistema genera una alerta para reposición. Esta decisión ayuda a evitar quiebres de stock y mejora la gestión del minimarket.
Uso de gestor visual de anaqueles	El sistema no solo controla productos y ventas, sino también la ubicación física de los productos. El gestor visual facilita la organización, búsqueda y reposición dentro del minimarket.
Generación de reportes	Los reportes de ventas, inventario, rotación y movimientos apoyan la toma de decisiones del administrador. Permiten identificar productos con mayor demanda y necesidades de reposición.
Uso de tablas normalizadas	Las entidades principales se separan en tablas específicas para evitar duplicidad de datos y mejorar la integridad del

Decisión	Justificación
	modelo. Esto permite mantener relaciones claras entre usuarios, productos, ventas, boletas, anaqueles y alertas.
Exclusión de pagos en línea	El alcance del proyecto se centra en consulta de productos, gestión de inventario, ventas en caja y control de anaqueles. No se incluyen pagos en línea para mantener el proyecto viable y coherente con los objetivos definidos.
Exclusión de delivery	No se incluye reparto ni seguimiento de pedidos porque el sistema está orientado al funcionamiento interno del minimarket y a la consulta previa de productos por parte del cliente.

12. VALIDACIÓN DEL DISEÑO

- Se verificó que los requisitos principales del sistema están cubiertos por los módulos definidos en el diseño técnico. Las funciones de búsqueda de productos, consulta de disponibilidad, registro de ventas, actualización de stock, gestión de inventario, alertas y reportes cuentan con componentes específicos dentro del sistema.
- Los casos de uso tienen soporte técnico mediante componentes, servicios internos, clases y tablas de base de datos. Por ejemplo, el caso de uso “Registrar venta” se apoya en el Módulo de Ventas, Servicio de Ventas, Servicio de Inventario, las clases Venta y DetalleVenta, y las tablas venta y detalle_venta.
- La base de datos es coherente con las necesidades del sistema, ya que incluye entidades relacionadas con usuarios, roles, productos, categorías, anaqueles, inventario, ventas, detalles de venta, boletas, alertas y reportes.
- El modelo relacional mantiene consistencia mediante claves primarias y claves foráneas, permitiendo relacionar correctamente productos con categorías, productos con anaqueles, ventas con detalles de venta, ventas con boletas y productos con alertas de stock.
- La normalización de la base de datos permite reducir redundancia y organizar la información de forma adecuada. Las tablas separan correctamente los datos de usuarios, productos, ventas, inventario y reportes, evitando duplicación innecesaria.
- La arquitectura propuesta es viable porque utiliza una estructura por capas: capa de presentación, capa de lógica de negocio y capa de datos. Esta separación facilita el mantenimiento, la escalabilidad y la implementación del sistema.
- La capa de presentación está soportada por Angular y Bootstrap, permitiendo una interfaz web responsive para cliente, trabajador/cajero y administrador.



- La capa de lógica de negocio está soportada por Node.js y Express.js, mediante una API REST encargada de recibir peticiones, validar datos, ejecutar procesos y comunicarse con la base de datos.
- La capa de datos está soportada por PostgreSQL, lo que permite almacenar la información del sistema de manera estructurada, relacional y consistente.
- Los diagramas de secuencia validan el comportamiento dinámico del sistema, mostrando cómo interactúan los actores con el frontend, la API REST, los servicios internos, los repositorios y la base de datos.
- El diseño de red soporta el sistema porque utiliza una topología cliente-servidor lógica, donde los clientes acceden mediante navegador web, el servidor de aplicaciones procesa las solicitudes y el servidor de base de datos almacena la información.
- Las conexiones definidas son coherentes con la arquitectura: el cliente se comunica con el servidor mediante HTTPS/API REST, y el backend se comunica con PostgreSQL mediante SQL sobre el puerto correspondiente.
- El diagrama de despliegue más red demuestra que la infraestructura lógica permite relacionar correctamente cliente, red lógica, servidor de aplicaciones, base de datos y servicios externos como correo, copias de seguridad o impresión.
- No se identifican componentes aislados, ya que cada módulo del sistema se relaciona con casos de uso, clases y tablas necesarias para su funcionamiento.
- En conclusión, el diseño técnico del sistema es coherente, viable y completo, ya que cubre los requisitos funcionales, brinda soporte técnico a los casos de uso, presenta una base de datos organizada y propone una arquitectura y red lógica adecuadas para la implementación del sistema web de gestión de inventario y ventas para minimarkets.